Lab 19 — Network Segmentation and Security Zones

> **SY0-701 Objective 3.2** — Given a scenario, apply security principles to secure enterprise infrastructure
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Build a screened subnet (DMZ), an internal zone and a management zone, prove that a flat network allows lateral movement, then enforce a default-deny zone policy and verify it.

You'll build: Three isolated layer 3 zones in Linux network namespaces behind a host router, with an iptables zone policy, packet counters and dropped-traffic logs.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1.

Background

- A **security zone** groups assets that share one trust level. Traffic crossing a zone boundary must pass an enforcement point — this is **device placement** in practice.
- The **screened subnet** (formerly DMZ) hosts internet-facing services. It may be reached from outside and from internal clients, but it must never initiate connections into the internal zone.
- A **management zone** holds the **jump server**: administrators connect there first and only that zone may reach every other zone for administration.
- **Default deny** plus explicit allow rules shrinks the **attack surface** and stops **lateral (east-west) movement**, reducing the blast radius of a compromised DMZ host.
- Each Linux network namespace has its own interfaces, routes and firewall — the same isolation a VLAN plus a router gives you on real switches.

Steps

1. Install the tools

Most tools are pre-installed; this makes sure ping, netcat and curl are present.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y iproute2 iptables iputils-ping netcat-openbsd curl python3
```

2. Create three zone namespaces

Each namespace is a separate network stack: `dmz` (screened subnet), `internal` and `mgmt`.

```
for z in dmz internal mgmt; do ip netns add $z; done
ip netns list
```

3. Wire each zone to the host router with veth pairs

The host acts as the router between zones (10.0.10.0/24 DMZ, 10.0.20.0/24 internal, 10.0.30.0/24 management). A temporary rule allows all inter-zone forwarding so you can see the flat network first.

```
declare -A NET=([dmz]=10 [internal]=20 [mgmt]=30)
for z in dmz internal mgmt; do
  n=${NET[$z]}
  ip link add veth-$z type veth peer name veth-$z-r
  ip link set veth-$z netns $z
  ip addr add 10.0.$n.1/24 dev veth-$z-r && ip link set veth-$z-r up
  ip netns exec $z ip addr add 10.0.$n.2/24 dev veth-$z
  ip netns exec $z ip link set veth-$z up
  ip netns exec $z ip link set lo up
  ip netns exec $z ip route add default via 10.0.$n.1
done
sysctl -w net.ipv4.ip_forward=1
iptables -I FORWARD 1 -i veth-+ -o veth-+ -j ACCEPT   # temporary: flat network
ip -br addr | grep veth
```

4. Start services and prove the flat network allows lateral movement

Publish a web server in the DMZ and a "database" port in the internal zone. With no policy, a compromised DMZ host can reach the database.

```
# a plain TCPServer avoids http.server's slow reverse-DNS lookup inside the namespace
ip netns exec dmz python3 -c 'import http.server as h, socketserver as s;
s.TCPServer(("10.0.10.2", 80), h.SimpleHTTPRequestHandler).serve_forever()' >/dev/null
2>&1 &
ip netns exec internal nc -lk 5432 >/dev/null 2>&1 &
sleep 1
ip netns exec internal curl -s -o /dev/null -w "internal -> DMZ web: HTTP %{http_code}\n"
http://10.0.10.2/
ip netns exec dmz nc -zv -w2 10.0.20.2 5432
ip netns exec dmz ping -c1 -W1 10.0.30.2 >/dev/null && echo "DMZ -> mgmt ping: reachable
(bad)"
```

5. Apply a default-deny zone policy

Replace the temporary rule with a dedicated `ZONES` chain: return traffic is allowed, the management zone may reach everything, internal clients may reach only the DMZ web port, and everything else is logged and dropped.

```
iptables -D FORWARD -i veth-+ -o veth-+ -j ACCEPT
iptables -N ZONES 2>/dev/null || iptables -F ZONES
iptables -A ZONES -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A ZONES -s 10.0.30.0/24 -m comment --comment "mgmt/jump server to any" -j ACCEPT
iptables -A ZONES -s 10.0.20.0/24 -d 10.0.10.0/24 -p tcp --dport 80 -m comment --comment
"internal to DMZ web" -j ACCEPT
iptables -A ZONES -m limit --limit 10/min -j LOG --log-prefix "ZONE-DROP "
iptables -A ZONES -m comment --comment "default deny" -j DROP
iptables -I FORWARD 1 -i veth-+ -o veth-+ -j ZONES
iptables -L ZONES -n --line-numbers
```

6. Re-test the screened-subnet policy

Every allowed path still works and every forbidden path now fails.

```
echo "internal -> DMZ web (allow):"; ip netns exec internal curl -s -o /dev/null -w "
HTTP %{http_code}\n" http://10.0.10.2/
echo "internal -> DMZ ping (deny, not a published service):"; ip netns exec internal ping
-c1 -W1 10.0.10.2 >/dev/null || echo "  blocked"
echo "DMZ -> internal DB (deny, lateral movement):"; ip netns exec dmz nc -zv -w2
10.0.20.2 5432 2>&1 | sed 's/^/  /'
echo "DMZ -> mgmt (deny):"; ip netns exec dmz ping -c1 -W1 10.0.30.2 >/dev/null || echo "
blocked"
echo "mgmt -> DMZ and internal (allow, jump server):"
ip netns exec mgmt ping -c1 -W1 10.0.10.2 >/dev/null && echo "  DMZ reachable"
ip netns exec mgmt nc -zv -w2 10.0.20.2 5432 2>&1 | sed 's/^/  /'
```

```
internal -> DMZ web (allow):
  HTTP 200
internal -> DMZ ping (deny, not a published service):
  blocked
DMZ -> internal DB (deny, lateral movement):
  nc: connect to 10.0.20.2 port 5432 (tcp) timed out: Operation now in progress
DMZ -> mgmt (deny):
  blocked
mgmt -> DMZ and internal (allow, jump server):
  DMZ reachable
  Connection to 10.0.20.2 5432 port [tcp/postgresql] succeeded!
```

7. Read the counters and dropped-packet logs

Counters prove which rules matched; the log gives the evidence a SOC analyst would forward to the SIEM.

```
iptables -L ZONES -n -v --line-numbers
dmesg 2>/dev/null | grep ZONE-DROP | tail -3
```

Test it

```
ip netns exec internal curl -s -o /dev/null -w "%{http_code}\n" http://10.0.10.2/
ip netns exec dmz nc -z -w2 10.0.20.2 5432 && echo "FAIL: lateral path open" || echo
"PASS: DMZ cannot reach internal"
```

Expected: `200`, then `PASS: DMZ cannot reach internal`. The `default deny` rule in `iptables -L ZONES -n -v` shows a non-zero packet count.

Troubleshooting

Symptom	Fix
`Cannot create namespace file ... File exists`	The lab was run before; continue from step 4, or start a fresh playground.
`internal -> DMZ web: HTTP 000`	The DMZ web server is not running; rerun the `python3 -c ... TCPServer` line from step 4 and wait a second.

Symptom	Fix
Step 4 shows nothing reachable	Docker may have set the FORWARD policy to DROP; confirm the temporary rule exists with `iptables -S FORWARD | head`.
No `ZONE-DROP` lines in dmesg	Kernel logging may be restricted in the playground; rely on the packet counters from `iptables -L ZONES -n -v`.

Challenge

Add a fourth zone, `iot` (10.0.40.0/24), that may only reach a single update server in the DMZ on TCP 443, and nothing else. Write the rule, then prove from the `iot` namespace that the internal zone is unreachable.

Reflection

The DMZ web server is compromised. Using your counters, explain how the zone policy limited the attacker's blast radius, and which rule implements the jump server concept.

Exam tips

- A screened subnet (DMZ) may accept connections but must not initiate them into the internal network.
- Segmentation limits lateral movement; a VLAN alone is not a security control until a firewall polices traffic between VLANs.
- A jump server concentrates and logs administrative access from a dedicated management zone.
- Default deny plus explicit allow is the secure baseline; the last rule is an implicit or explicit deny.

References

- NIST SP 800-207 Zero Trust Architecture — https://csrc.nist.gov/pubs/sp/800/207/final
- NIST SP 800-41 Rev. 1 Guidelines on Firewalls and Firewall Policy — https://csrc.nist.gov/pubs/sp/800/41/r1/final
- CIS Critical Security Controls v8 (Control 12 Network Infrastructure Management) — https://www.cisecurity.org/controls
- ip-netns manual — https://man7.org/linux/man-pages/man8/ip-netns.8.html

---